# Jetson Nano GPU Guide

Using the 128-core Maxwell GPU on the Jetson Nano for Robot Software

DIY Challenge Repo · ROS 2 Humble · CUDA 11.x · TensorRT

| | |
|---|---|
| **Target Platform** | NVIDIA Jetson Nano 4 GB (JetPack 5.x) |
| **GPU Architecture** | Maxwell — 128 CUDA cores, 921 MHz |
| **CUDA Version** | 11.4 (from JetPack 5.x) |
| **TensorRT Version** | 8.x (from JetPack 5.x) |
| **ROS 2 Version** | Humble |
| **Current GPU Usage** | 0% — navigation stack is CPU-only (this guide changes that) |

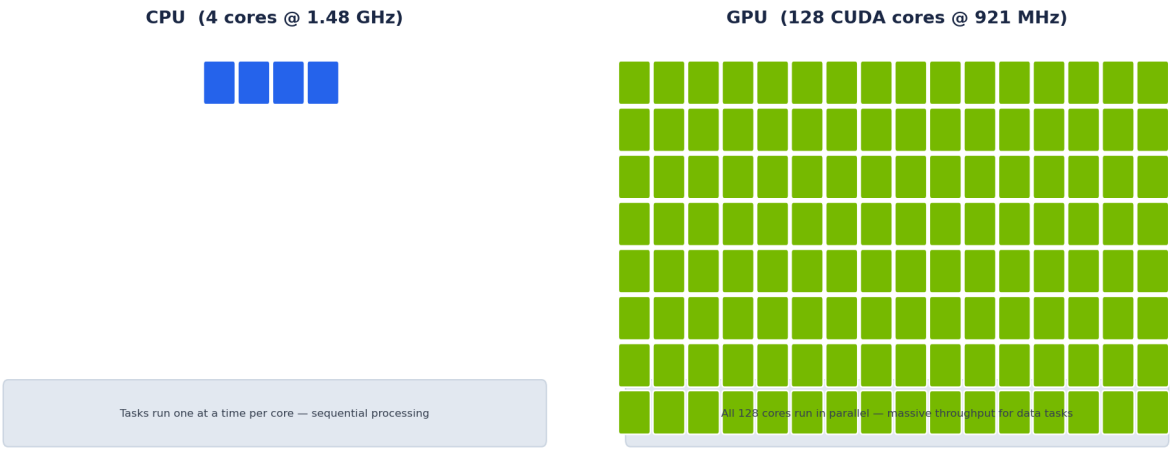
# 1. What Is the Jetson Nano GPU and When Does It Help?

## 1.1 Hardware overview

The Jetson Nano contains a **128-core NVIDIA Maxwell GPU** on the same die as the 4-core ARM Cortex-A57 CPU. Unlike a gaming GPU that has thousands of cores, the Maxwell is a small but capable GPU designed for embedded inference workloads. The CPU and GPU share the same physical RAM (4 GB LPDDR4) — this means data does not have to travel over a PCIe bus; it can be passed between CPU and GPU with near-zero copy overhead.

## 1.2 CPU vs GPU — the fundamental difference

A CPU is optimised for running a small number of complex, sequential tasks quickly. A GPU is optimised for running a very large number of simple tasks simultaneously. The diagram below illustrates this:




**Practical implication**: if your algorithm processes each lidar point, each image pixel, or each neural-network weight in sequence (like a Python for-loop), the GPU cannot help. If the same operation can be applied to thousands of data elements *at the same time*, the GPU can be 10-100× faster than the CPU.

## 1.3 What currently runs on the CPU in this repo

- **FAST-LIO2** — Eigen matrix math + ikd-Tree on CPU (4 threads)
- **LIO-SAM** — GTSAM factor graphs + OpenMP CPU threads
- **Nav2 (MPPI planner)** — trajectory sampling on CPU
- **EKF nodes** — single-threaded Kalman filter math
- **cmd_vel_mux** — simple topic arbitration, trivially CPU
- **estop_controller** — heartbeat watchdog, trivially CPU

NOTE: None of these are GPU bottlenecked — they are compute-bound on the CPU in ways that map poorly to GPU parallelism. Adding GPU code to these nodes would not meaningfully improve performance and would add significant complexity.

## 1.4 Where the GPU CAN help in a robot like this

| Use Case | GPU Benefit | Complexity | Typical Speedup |
|---|---|---|---|
| Object detection (YOLO) | TensorRT runs neural network inference | Medium | 10-50× |
| Point cloud filtering / downsampling | CUDA parallelises voxel grid on all points at once | Medium | 3-10× |

| | | | |
|---|---|---|---|
| Stereo depth from D435i images | CUDA Semi-Global Matching | High | 5-20× |
| Image preprocessing (resize, normalise) | CUDA/cuDNN accelerates for DNN pipelines | Low (via cuDNN) | 2-5× |
| Nav2 MPPI trajectory sampling | Experimental GPU MPPI in Nav2 Jazzy+ | High (Nav2 Humble does not support it) | 2-8× |

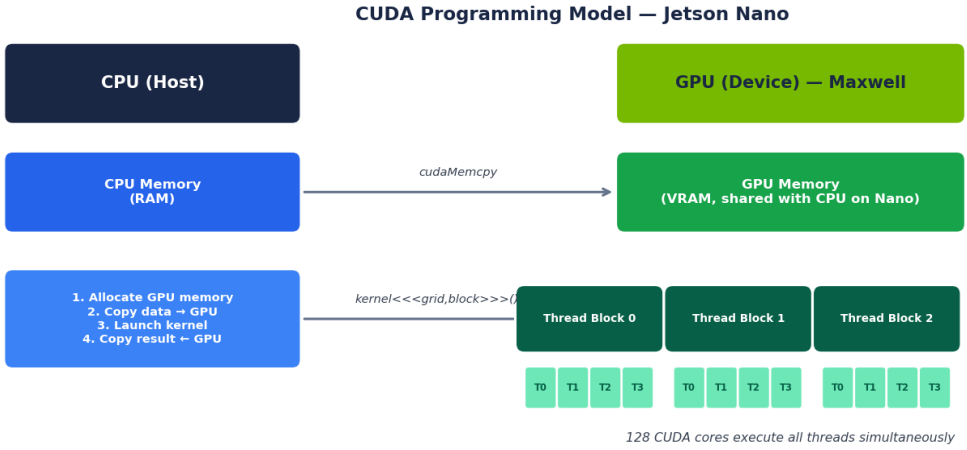
# 2. CUDA Programming Basics

CUDA (Compute Unified Device Architecture) is NVIDIA's programming model for GPU computing. You write special C++ functions called **kernels** that run on the GPU. The CUDA compiler (nvcc) compiles them alongside your normal C++ code.

## 2.1 Key concepts

- **Host**: the CPU and its RAM. Your normal C++ code runs here.
- **Device**: the GPU and its memory. CUDA kernels run here.
- **Kernel**: a GPU function marked __global__. Called from CPU, runs on GPU.
- **Thread**: the smallest unit of work on the GPU. Thousands run simultaneously.
- **Block**: a group of threads that share fast on-chip memory.
- **Grid**: the collection of all blocks for one kernel launch.
- **cudaMalloc / cudaFree**: allocate/free GPU memory.
- **cudaMemcpy**: copy data between CPU and GPU memory.

## 2.2 CUDA programming model diagram




## 2.3 Minimal working CUDA example

The example below adds two arrays on the GPU — the GPU equivalent of a simple for-loop. This is the smallest useful CUDA program.

vector_add.cu — minimal CUDA kernel example

```
// vector_add.cu
#include <stdio.h>

// The __global__ keyword marks this as a GPU kernel.
// Each GPU thread runs this function once with a different blockIdx/threadIdx.
__global__ void vectorAdd(float* A, float* B, float* C, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;  // which element am I?
    if (i < n) {
        C[i] = A[i] + B[i];   // each thread adds one pair of elements
    }
}

int main() {
    int n = 1024;
    size_t bytes = n * sizeof(float);
```

```
    // 1. Allocate CPU (host) memory
    float *h_A = (float*)malloc(bytes);
    float *h_B = (float*)malloc(bytes);
    float *h_C = (float*)malloc(bytes);
    for (int i = 0; i < n; i++) { h_A[i] = i; h_B[i] = i * 2; }

    // 2. Allocate GPU (device) memory
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, bytes);
    cudaMalloc(&d_B, bytes);
    cudaMalloc(&d_C, bytes);

    // 3. Copy data from CPU → GPU
    cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);

    // 4. Launch 1024 threads in blocks of 256
    int blockSize = 256;
    int gridSize  = (n + blockSize - 1) / blockSize;   // = 4 blocks
    vectorAdd<<<gridSize, blockSize>>>(d_A, d_B, d_C, n);

    // 5. Copy result GPU → CPU
    cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost);

    printf("C[0]=%f  C[1023]=%f\n", h_C[0], h_C[1023]);  // 0, 3069

    // 6. Free GPU memory
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    free(h_A);     free(h_B);     free(h_C);
    return 0;
}
```

**Compile and run on Jetson Nano**

```
# nvcc is the CUDA compiler (comes with JetPack)
nvcc -o vector_add vector_add.cu
./vector_add
# Output: C[0]=0.000000  C[1023]=3069.000000
```

## 2.4 What changes vs normal C++

| Concept | Normal C++ | CUDA equivalent |
|---|---|---|
| Function on GPU | — (not possible) | __global__ void kernel(...) |
| Allocate memory | malloc() / new | cudaMalloc() |
| Free memory | free() / delete | cudaFree() |
| Copy data | memcpy() | cudaMemcpy() |
| Launch function | func(args) | func<<>>(args) |
| File extension | .cpp | .cu |
| Compiler | g++ | nvcc |

# 3. TensorRT — GPU Inference Without Writing CUDA

For robot applications, **you almost never need to write raw CUDA kernels**. NVIDIA's **TensorRT** library is a high-performance deep learning inference engine that: (1) takes a trained neural network (ONNX format), (2) optimises it automatically for the specific GPU it is running on, and (3) runs it as fast as possible — all without you writing a single line of CUDA. This is the practical way to use the Jetson GPU.

## 3.1 The TensorRT workflow

- **Train** your model (e.g. YOLOv8) on a desktop/cloud GPU using PyTorch
- **Export** to ONNX format (model.export(format="onnx"))
- **Convert** on the Jetson: TensorRT parses the ONNX and builds a .engine file optimised for the Maxwell GPU (takes ~5 min, done once)
- **Run** the .engine file at inference time — TensorRT handles all GPU memory management, kernel selection, and parallelism automatically

## 3.2 Install TensorRT (already in JetPack)

Verify TensorRT is installed from JetPack

```
# TensorRT is pre-installed with JetPack 5.x. Verify:
python3 -c "import tensorrt; print(tensorrt.__version__)"
# Expected: 8.x.x

dpkg -l | grep -i tensorrt    # list all TensorRT packages

# Also check the C++ headers are present:
ls /usr/include/x86_64-linux-gnu/NvInfer.h 2>/dev/null || \
ls /usr/include/aarch64-linux-gnu/NvInfer.h  # Jetson is aarch64
```

## 3.3 Install torch2trt and onnxruntime-gpu (optional helpers)

Install Python TensorRT helpers

```
# onnxruntime-gpu — run ONNX models on GPU without converting to TensorRT engine
pip3 install onnxruntime-gpu

# torch2trt — convert PyTorch models directly to TensorRT (skips ONNX step)
git clone https://github.com/NVIDIA-AI-IOT/torch2trt
cd torch2trt && sudo python3 setup.py install
```

## 3.4 Convert YOLOv8 to TensorRT — step by step

This section walks through adding GPU-based object detection to the robot. The detector publishes /detected_objects (vision_msgs/Detection2DArray) which Nav2 collision monitor can subscribe to as a dynamic obstacle source.

Step 1: Export YOLOv8 to ONNX (on laptop/desktop)

```
# Install ultralytics on your laptop (not Jetson — uses more RAM)
pip install ultralytics

# Download a pretrained model and export to ONNX
python3 - << 'EOF'
from ultralytics import YOLO
model = YOLO("yolov8n.pt")          # 'n' = nano (smallest, fastest)
model.export(format="onnx",
             imgsz=640,
             simplify=True,
             dynamic=False)         # saves yolov8n.onnx
EOF
```

```
# Copy yolov8n.onnx to the Jetson:
scp yolov8n.onnx ubuntu@jetson.local:~/models/
```

#### Step 2: Build TensorRT engine on the Jetson (run once)

```
# SSH into Jetson first, then:
mkdir -p ~/models && cd ~/models

# Use trtexec (comes with TensorRT) to build the engine
/usr/src/tensorrt/bin/trtexec \
    --onnx=yolov8n.onnx \
    --saveEngine=yolov8n_fp16.engine \
    --fp16 \                          # half-precision: 2x speed on Maxwell GPU
    --workspace=1024                   # MB of GPU workspace

# This takes 3-8 minutes. The output .engine file is hardware-specific.
# Do NOT copy the .engine file between different machines.
```

#### Step 3: Run inference from Python (test)

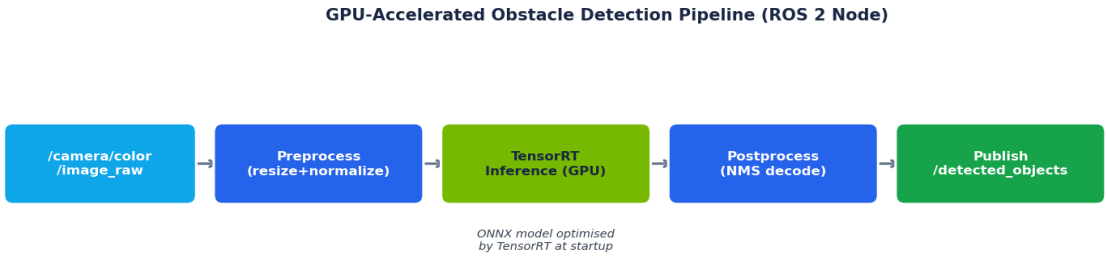
```
import tensorrt as trt
import numpy as np
import pycuda.driver as cuda
import pycuda.autoinit
import cv2

# Load the engine
logger = trt.Logger(trt.Logger.WARNING)
with open("/home/ubuntu/models/yolov8n_fp16.engine", "rb") as f:
    runtime = trt.Runtime(logger)
    engine  = runtime.deserialize_cuda_engine(f.read())

context = engine.create_execution_context()

# Allocate GPU buffers (one per input/output binding)
inputs, outputs, bindings = [], [], []
stream = cuda.Stream()
for i in range(engine.num_io_tensors):
    name  = engine.get_tensor_name(i)
    shape = engine.get_tensor_shape(name)
    dtype = trt.nptype(engine.get_tensor_dtype(name))
    size  = int(np.prod(shape))
    host_mem   = cuda.pagelocked_empty(size, dtype)
    device_mem = cuda.mem_alloc(host_mem.nbytes)
    bindings.append(int(device_mem))
    if engine.get_tensor_mode(name) == trt.TensorIOMode.INPUT:
        inputs.append({'host': host_mem, 'device': device_mem})
    else:
        outputs.append({'host': host_mem, 'device': device_mem})

# Run one inference on a test image
img = cv2.imread("test.jpg")
img = cv2.resize(img, (640, 640))
img = img.astype(np.float32) / 255.0
img = img.transpose(2, 0, 1)         # HWC -> CHW
np.copyto(inputs[0]['host'], img.ravel())

# Copy input CPU -> GPU, run, copy output GPU -> CPU
cuda.memcpy_htod_async(inputs[0]['device'], inputs[0]['host'], stream)
context.execute_async_v2(bindings, stream.handle, None)
cuda.memcpy_dtoh_async(outputs[0]['host'], outputs[0]['device'], stream)
stream.synchronize()

print("Inference complete. Output shape:", outputs[0]['host'].shape)
```

WARNING: The TensorRT .engine file is built for a specific GPU architecture AND TensorRT version. Always build it on the Jetson Nano itself, not on your laptop. If you update JetPack, rebuild the engine.

# 4. Adding a GPU Inference Node to the ROS 2 Stack

The practical approach is to wrap the TensorRT inference in a standard ROS 2 Python node. The node subscribes to the camera image topic, runs GPU inference, and publishes detections. No changes to FAST-LIO2 or Nav2 are required.

## 4.1 Pipeline overview




## 4.2 Create the ROS 2 package

Create diy_obstacle_detector package

```
cd ~/ros2_ws/src/DIY-Challenge-Repo/src
ros2 pkg create --build-type ament_python diy_obstacle_detector \
    --dependencies rclpy sensor_msgs vision_msgs cv_bridge
```

## 4.3 GPU inference node (complete implementation)

src/diy_obstacle_detector/diy_obstacle_detector/detector_node.py

```
#!/usr/bin/env python3
'''
GPU-accelerated YOLOv8 obstacle detector - ROS 2 Humble node.
Subscribes to /camera/color/image_raw, runs TensorRT inference on the
Jetson GPU, publishes /detected_objects (vision_msgs/Detection2DArray).
'''
import numpy as np
import cv2
import tensorrt as trt
import pycuda.driver as cuda
import pycuda.autoinit          # initialises CUDA context automatically

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray, Detection2D, BoundingBox2D
from cv_bridge import CvBridge


class DetectorNode(Node):
    def __init__(self):
        super().__init__('diy_obstacle_detector')

        # Parameters — set via launch file or ros2 param set
        self.declare_parameter('engine_path',  '/home/ubuntu/models/yolov8n_fp16.engine')
        self.declare_parameter('conf_threshold', 0.45)
        self.declare_parameter('input_size',    640)

        engine_path  = self.get_parameter('engine_path').value
```

```
        self.conf_th = self.get_parameter('conf_threshold').value
        self.inp_sz  = self.get_parameter('input_size').value

        # Load TensorRT engine
        self.get_logger().info(f'Loading TensorRT engine: {engine_path}')
        logger = trt.Logger(trt.Logger.WARNING)
        with open(engine_path, 'rb') as f:
            runtime = trt.Runtime(logger)
            self.engine = runtime.deserialize_cuda_engine(f.read())
        self.context = self.engine.create_execution_context()

        # Allocate pinned CPU buffers and GPU buffers for TensorRT IO
        self.inputs, self.outputs, self.bindings = [], [], []
        self.stream = cuda.Stream()
        for i in range(self.engine.num_io_tensors):
            name  = self.engine.get_tensor_name(i)
            shape = self.engine.get_tensor_shape(name)
            dtype = trt.nptype(self.engine.get_tensor_dtype(name))
            size  = int(np.prod(shape))
            host_mem   = cuda.pagelocked_empty(size, dtype)
            device_mem = cuda.mem_alloc(host_mem.nbytes)
            self.bindings.append(int(device_mem))
            entry = {'host': host_mem, 'device': device_mem, 'shape': shape}
            if self.engine.get_tensor_mode(name) == trt.TensorIOMode.INPUT:
                self.inputs.append(entry)
            else:
                self.outputs.append(entry)

        self.bridge = CvBridge()
        self.sub    = self.create_subscription(
            Image, '/camera/color/image_raw', self.image_cb, 10)
        self.pub    = self.create_publisher(
            Detection2DArray, '/detected_objects', 10)

        self.get_logger().info('Detector ready — GPU inference active.')

    def image_cb(self, msg: Image):
        # Convert ROS Image → OpenCV BGR
        frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
        h_orig, w_orig = frame.shape[:2]

        # Preprocess: resize → normalise → CHW layout
        inp = cv2.resize(frame, (self.inp_sz, self.inp_sz))
        inp = inp.astype(np.float32) / 255.0
        inp = inp.transpose(2, 0, 1)   # (H, W, C) -> (C, H, W)

        # Copy to pinned host buffer and run GPU inference
        np.copyto(self.inputs[0]['host'], inp.ravel())
        cuda.memcpy_htod_async(self.inputs[0]['device'],
                               self.inputs[0]['host'], self.stream)
        self.context.execute_async_v2(self.bindings, self.stream.handle)
        cuda.memcpy_dtoh_async(self.outputs[0]['host'],
                               self.outputs[0]['device'], self.stream)
        self.stream.synchronize()   # wait for GPU to finish

        # Decode YOLOv8 output: shape (1, 84, 8400) → filter by confidence
        raw = self.outputs[0]['host'].reshape(1, 84, -1)[0]  # (84, 8400)
        raw = raw.T                                           # (8400, 84)
        scores = raw[:, 4:].max(axis=1)
        mask   = scores > self.conf_th
        raw    = raw[mask]
```

```
        scores = scores[mask]

        det_array = Detection2DArray()
        det_array.header = msg.header
        sx = w_orig / self.inp_sz
        sy = h_orig / self.inp_sz

        for row, score in zip(raw, scores):
            cx, cy, w, h = row[:4]
            det = Detection2D()
            det.bbox.center.position.x = float(cx * sx)
            det.bbox.center.position.y = float(cy * sy)
            det.bbox.size_x = float(w * sx)
            det.bbox.size_y = float(h * sy)
            det_array.detections.append(det)

        self.pub.publish(det_array)


def main(args=None):
    rclpy.init(args=args)
    node = DetectorNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

## 4.4 Add to challenge_master.launch.py (optional)

Integration snippet for challenge_master.launch.py

```
# Add this block inside generate_launch_description():
from launch_ros.actions import Node as LaunchNode

obstacle_detector = LaunchNode(
    package='diy_obstacle_detector',
    executable='detector_node',
    name='diy_obstacle_detector',
    parameters=[{
        'engine_path':    '/home/ubuntu/models/yolov8n_fp16.engine',
        'conf_threshold': 0.45,
        'input_size':     640,
    }],
    remappings=[('/camera/color/image_raw', '/camera/color/image_raw')],
)

# Then add obstacle_detector to the return list in generate_launch_description()
```

## 4.5 Connect detections to Nav2 collision monitor

The Nav2 Collision Monitor can subscribe to /detected_objects as a dynamic obstacle source, causing the robot to slow down or stop when objects are detected in the camera field of view.

Add to collision_monitor_params.yaml

```
collision_monitor:
  ros__parameters:
    polygon_action: "stop"
    observation_sources: [lidar_scan, camera_detections]

    # Existing lidar source ...
```

```
camera_detections:
  type: "Detection2D"
  topic: "/detected_objects"
  min_height: 0.1
  max_height: 2.0
```

# 5. GPU-Accelerated Point Cloud Processing (CUDA)

If FAST-LIO2 is struggling with processing speed at high lidar frequencies, a GPU voxel-grid filter applied upstream can reduce the point count from ~65,000 QT64 points to ~5,000 before FAST-LIO2 ever sees them. This is one of the few places where writing a small amount of CUDA code directly provides a meaningful benefit.

## 5.1 Option A: use CUDAlib (no raw CUDA needed)

**Install and use cuda-pcl for point cloud filtering**

```
# cuda-pcl is NVIDIA's GPU-accelerated PCL equivalent
git clone https://github.com/NVIDIA-AI-IOT/cuda-pcl.git
cd cuda-pcl && mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j4 && sudo make install

# Use CudaVoxelGrid in your ROS 2 node instead of pcl::VoxelGrid:
# #include <cudaVoxelGrid.h>
# CudaVoxelGrid<pcl::PointXYZI> vg;
# vg.setLeafSize(0.1f, 0.1f, 0.1f);
# vg.setInputCloud(input_cloud);
# vg.filter(*output_cloud);   // runs on GPU
```

## 5.2 Option B: write a minimal CUDA voxel filter kernel

The following is a conceptual skeleton showing how a GPU voxel filter works. Each CUDA thread checks whether a point belongs to an occupied voxel cell and keeps only one representative per cell.

**voxel_filter_kernel.cu — conceptual skeleton**

```
// Each thread processes one input point.
// Atomically marks a voxel as occupied; only the first thread to claim
// a voxel writes its point to the output.
__global__ void voxelFilterKernel(
    const float4* __restrict__ input,   // x, y, z, intensity
    float4*       output,
    int*          output_count,
    int           n_points,
    float         leaf_size)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n_points) return;

    float4 p = input[idx];

    // Compute voxel bin index (simple hash)
    int ix = (int)(p.x / leaf_size);
    int iy = (int)(p.y / leaf_size);
    int iz = (int)(p.z / leaf_size);

    // Only ONE representative per voxel reaches the output.
    // (Full implementation uses a hash table in shared memory)
    int slot = atomicAdd(output_count, 1);
    output[slot] = p;
}

// Host call:
// dim3 block(256);
// dim3 grid((n_points + 255) / 256);
// voxelFilterKernel<<<grid, block>>>(d_input, d_output, d_count, n, 0.1f);
```

NOTE: For most teams, Option A (cuda-pcl) is the right choice — complex hash-table voxel grids are hard to implement efficiently in raw CUDA. Use cuda-pcl unless you have a specific reason to write your own kernel.

# 6. Verify GPU Usage and Monitor Performance

### Check that the GPU is actually being used

```
# Install jtop (Jetson system monitor — install once)
sudo pip3 install jetson-stats
sudo systemctl restart jtop.service  # needed after first install

# Run jtop — provides a live dashboard with CPU, GPU, RAM, temp
jtop

# Key column in jtop to watch:
#   GPU %  — should jump from ~0% to 30-80% when inference is running
#   RAM    — GPU uses system RAM; watch for OOM during model load
#   Temp   — keep GPU below 75°C

# Alternative: tegrastats (built-in, no install needed)
tegrastats --interval 500
# Look for:  GPU xxx%  in the output line
```

### Benchmark TensorRT inference speed

```
# trtexec can benchmark the engine directly (before adding to ROS 2):
/usr/src/tensorrt/bin/trtexec \
    --loadEngine=/home/ubuntu/models/yolov8n_fp16.engine \
    --batch=1 \
    --iterations=200

# Expected output for YOLOv8n FP16 on Jetson Nano:
#   mean: ~12-18 ms/inference  →  ~55-85 FPS
# Compare to CPU (onnxruntime, no GPU):
#   mean: ~180-300 ms/inference  →  ~3-5 FPS
```

### Python micro-benchmark — time one inference call

```
import time, numpy as np, pycuda.driver as cuda, pycuda.autoinit, tensorrt as trt

# (load engine and create context as shown in Section 4.3)

dummy_input = np.random.rand(3, 640, 640).astype(np.float32)
np.copyto(inputs[0]['host'], dummy_input.ravel())

# Warm-up (first call initialises caches)
for _ in range(5):
    cuda.memcpy_htod_async(inputs[0]['device'], inputs[0]['host'], stream)
    context.execute_async_v2(bindings, stream.handle)
    cuda.memcpy_dtoh_async(outputs[0]['host'], outputs[0]['device'], stream)
    stream.synchronize()

# Timed runs
t0 = time.perf_counter()
N = 100
for _ in range(N):
    cuda.memcpy_htod_async(inputs[0]['device'], inputs[0]['host'], stream)
    context.execute_async_v2(bindings, stream.handle)
    cuda.memcpy_dtoh_async(outputs[0]['host'], outputs[0]['device'], stream)
    stream.synchronize()
print(f"Mean latency: {(time.perf_counter()-t0)/N*1000:.1f} ms")
print(f"Throughput:   {N/(time.perf_counter()-t0):.1f} FPS")
```

TIP: Always run jetson_clocks before benchmarking or running inference at competition. The GPU clock defaults to a lower frequency at boot and can be 2× slower without clock locking. See Section 5.7 of DIY_Challenge_Robot_Guide.pdf for the persistent systemd service that locks clocks at startup.

# 7. Quick-Reference Checklist

## One-time setup (do once on the Jetson)

- Verify JetPack version: cat /etc/nv_tegra_release
- Verify CUDA: nvcc --version → 11.4.x
- Verify TensorRT: python3 -c "import tensorrt; print(tensorrt.__version__)"
- Install jetson-stats: sudo pip3 install jetson-stats
- Create models directory: mkdir -p ~/models
- Export YOLOv8n to ONNX on laptop, copy to ~/models/yolov8n.onnx
- Build TensorRT engine on Jetson with trtexec --fp16
- Verify engine with trtexec benchmark → should show >50 FPS for YOLOv8n

## Every run (before launching the robot)

- sudo nvpmodel -m 0 — switch to MAXN (10W) power mode
- sudo jetson_clocks — lock CPU/GPU to max frequency
- jtop — confirm GPU clock is at maximum (921 MHz or close)
- Check GPU temperature is below 40°C at idle before a long run

## Troubleshooting GPU issues

| Problem | Likely Cause | Fix |
|---|---|---|
| GPU % stays at 0% in jtop | TensorRT engine not loaded / node not running | Check ros2 node list, check engine path parameter |
| CUDA out of memory error | Model too large for 4 GB shared RAM | Use yolov8n (nano) not larger variants; add 4 GB swap |
| trtexec shows <20 FPS on YOLOv8n | GPU clock not locked | Run sudo jetson_clocks, check nvpmodel -q shows mode 0 |
| Engine file fails to load | Engine built on different TensorRT version | Rebuild engine on THIS Jetson after any JetPack update |
| Inference latency spikes | Thermal throttling (GPU > 85°C) | Add heatsink/fan; check jtop Temp column |

## Further reading

- CUDA C++ Programming Guide: https://docs.nvidia.com/cuda/cuda-c-programming-guide/
- TensorRT Developer Guide: https://docs.nvidia.com/deeplearning/tensorrt/developer-guide/
- YOLOv8 on Jetson (NVIDIA blog): https://developer.nvidia.com/blog/real-time-object-detection-on-jetson-nano/
- Jetson Nano Developer Kit User Guide: https://developer.nvidia.com/embedded/learn/get-started-jetson-nano-devkit
- cuda-pcl (GPU point cloud library): https://github.com/NVIDIA-AI-IOT/cuda-pcl

# Appendix A — Why the Core Navigation Algorithms Are Not GPU-Compatible

This appendix answers the direct question: *"Should I try to run FAST-LIO2, LIO-SAM, Nav2, or the EKF on the Jetson GPU to improve performance?"* The short answer is **no**. This section explains the technical reasons why, algorithm by algorithm.

WARNING: Attempting to GPU-port these algorithms would add weeks of low-level CUDA engineering for zero measurable gain at competition scale. Do not attempt it. Use the GPU exclusively for the TensorRT inference pipeline described in §4.

## A.1 The GPU Fundamental Requirement

A GPU delivers speedup only when ALL of the following are true simultaneously:

- **Massively parallel**: thousands of identical, independent operations per frame
- **Regular memory access**: threads read consecutive addresses (coalesced access) — every cache miss in GPU global memory costs ~500 cycles
- **Uniform branching**: all threads in a warp execute the same path; divergence serialises the warp back to single-threaded speed
- **High arithmetic intensity**: many FLOPs per byte of data moved (otherwise the bottleneck is memory bandwidth, not raw compute)

Every core navigation algorithm in this repo fails at least one of these criteria. The table below summarises the verdict; the subsections explain the technical reasons.

| Algorithm | Primary Data Structure | Why GPU Cannot Help | Verdict |
|---|---|---|---|
| FAST-LIO2 | ikd-Tree (incremental k-d tree) | Pointer-chasing tree traversal; sequential iEKF updates | CPU only |
| LIO-SAM | GTSAM Bayes tree factor graph | Factor graph optimisation is inherently sequential; linearisation order matters | CPU only |
| Nav2 MPPI planner | Trajectory rollout array | GPU MPPI only in Nav2 Jazzy+; Humble CPU MPPI is well-tuned for <200 trajectories | CPU only (Humble) |
| EKF (robot_localization) | 6×6 or 15×15 state matrix | State vector too small; GPU launch overhead exceeds the entire matrix multiply time | CPU only |
| ndt_omp_ros2 | Voxel grid + normal distributions | OpenMP CPU threads already saturate available cores; GPU port exists but requires CUDA NDT library | CPU preferred |

## A.2 FAST-LIO2 — ikd-Tree is GPU-Hostile by Design

FAST-LIO2's scan-registration step is built around the **incremental k-d tree (ikd-Tree)**. This is a balanced binary search tree stored as a linked list of heap-allocated nodes. At each lidar scan, FAST-LIO2 performs:

- **Nearest-neighbour queries**: traverse from root to leaf, following pointers at each level
- **Incremental insertion/deletion**: rebalance sub-trees on new points
- **Iterated Extended Kalman Filter (iEKF) update**: repeated 3×3 or 6×6 matrix solves, each conditioned on the result of the previous

GPU memory architecture is optimised for *sequential, strided* access to large contiguous arrays — the opposite of pointer-following tree traversal. Every node fetch during a k-d tree query is an unpredictable

memory address. On GPU this causes a **global memory miss costing ~500 cycles** per node visit. With a typical tree depth of ~17 levels and ~5,000 nearest-neighbour queries per scan, that is 85,000 cache misses per frame — at 10 Hz lidar this means the GPU spends ~425 million stall cycles simply waiting for memory.

The iEKF loop is equally problematic: each linearisation step depends on the output of the previous step. This hard sequential dependency means only one CUDA thread can be active at a time — a GPU running a for-loop.

NOTE: FAST-LIO2 on CPU takes ~8–15 ms per scan on the Jetson Nano. A GPU port of the ikd-Tree nearest-neighbour step would likely be *slower* than the CPU version due to memory-bandwidth bottleneck, while requiring ~2,000 lines of CUDA kernel code to implement correctly.

## A.3 LIO-SAM — Factor Graph Optimisation is Sequential

LIO-SAM uses **GTSAM** to maintain a factor graph of robot poses. When new sensor measurements arrive, GTSAM performs iSAM2 (incremental Smoothing and Mapping) to update the graph. The iSAM2 algorithm:

- Identifies which variables in the Bayes tree are affected by the new factor
- Re-linearises those variables *in topological order* (parent before child)
- Back-substitutes in reverse topological order to recover the new estimate

Both the forward and backward passes enforce a strict ordering. There is no subset of these operations that can be done independently in parallel. GPU parallelism requires independence — GTSAM's entire design principle is the opposite.

Additionally, LIO-SAM already uses **OpenMP to parallelise feature extraction** across ring channels and the ground segmentation step. These CPU threads already saturate the Jetson Nano's 4-core ARM CPU. The factor graph optimisation itself typically takes <2 ms per keyframe; it is not a bottleneck.

## A.4 Nav2 MPPI — GPU Version Requires Nav2 Jazzy+

MPPI (Model Predictive Path Integral) trajectory sampling is, in principle, parallelisable — each sampled trajectory is independent. NVIDIA showed a GPU MPPI implementation in Nav2. **However:**

- The GPU MPPI controller in Nav2 was merged in **Nav2 Jazzy** (ROS 2 2024 LTS)
- This repo uses **ROS 2 Humble** — the GPU MPPI plugin does not exist here
- Backporting is non-trivial: it requires porting the entire CudaMPPI plugin and its deps to Humble
- At competition scale (indoor arena <20m), the default Humble CPU MPPI with ~50 trajectories is more than adequate — <10 ms per control cycle

NOTE: If this project ever migrates to ROS 2 Jazzy or Kilted, the GPU MPPI plugin becomes available as a drop-in replacement for the CPU planner. No code changes to the rest of the stack are needed at that point.

## A.5 EKF (robot_localization) — State Matrix Too Small

The EKF fuses IMU, wheel odometry, and SLAM pose into a single state estimate. The state vector is at most 15 elements (full 6-DOF with derivatives). The prediction step requires a 15×15 matrix multiply and a 15-element vector add. On CPU this takes **~1–5 microseconds**.

Launching a CUDA kernel has a fixed overhead of **~5–15 microseconds** just for the kernel launch + memory transfer, before any computation begins. Moving the EKF predict step to GPU would make it 5–15× *slower*, not faster.

GPU acceleration makes sense when the kernel runs for milliseconds (DNN inference, image processing). For microsecond-scale state updates, the overhead dominates.

## A.6 Final Recommendation

The table below is the definitive answer for where to invest GPU effort in a competition run with this software stack:

| Component | Run on GPU? | Action |
|---|---|---|
| FAST-LIO2 | No | Leave on CPU. Lock Jetson clocks with nvpmodel -m 0 + jetson_clocks to get max CPU frequency. |
| LIO-SAM | No | Leave on CPU. OpenMP already saturates cores. |
| Nav2 + EKF | No | Leave on CPU. Not a bottleneck at competition scale. |
| Point cloud downsampling (upstream of FAST-LIO2) | Optional | Use cuda-pcl voxel grid (§5) only if FAST-LIO2 is dropping scans due to CPU overload at high lidar frequency. |
| YOLOv8 obstacle detection | Not recommended (see Appendix B) | The existing lidar-based Nav2 Collision Monitor already handles competition obstacles robustly. YOLOv8 adds engineering risk with marginal benefit. See Appendix B for the full analysis. |

NOTE: ZERO changes to FAST-LIO2, LIO-SAM, Nav2, or EKF source code are needed or beneficial for GPU use. The current CPU-only navigation stack is the right architecture for this competition. See Appendix B for a full analysis of why YOLOv8 is not the preferred obstacle-avoidance approach.

# Appendix B — Why YOLOv8 Is Not Preferred for Obstacle Avoidance

The §4 TensorRT inference pipeline demonstrates that YOLOv8 *can* run on the Jetson GPU. This appendix explains why it should **not** replace or augment the existing Nav2 Collision Monitor for the competition environment. The current lidar-based approach is more robust, simpler, and better suited to the conditions under which this robot competes.

## B.1 What the Current Stack Already Does

The Nav2 Collision Monitor watches the raw Hesai lidar point cloud in real time. Two safety zones are configured around the robot:

- **PolygonStop** (35 cm ahead, 50 cm wide): any 3+ lidar points → immediate hard stop
- **PolygonSlow** (50 cm ahead, 60 cm wide): any 3+ lidar points → 40% speed reduction

This is evaluated at the full lidar frequency (10–20 Hz) with deterministic latency. No camera, no neural network, no GPU required. For any solid obstacle in the competition arena — boxes, walls, people's legs, cones, barriers — the lidar detects it reliably and the robot stops.

## B.2 Why the Lidar-Only Approach Is Superior for This Competition

| Property | Lidar Collision Monitor (current) | YOLOv8 + Camera |
|---|---|---|
| Detection principle | Physics — laser time-of-flight, independent of appearance | Pattern recognition — trained on labelled images |
| Lighting sensitivity | None — works in complete darkness | High — performance degrades with glare, shadows, overexposure |
| Unlabelled obstacle classes | Detects anything that reflects a laser pulse | Only detects classes present in training data |
| Latency | One lidar scan (~50–100 ms) — deterministic | Camera frame + TensorRT inference + ROS message (~30–80 ms, jitter) |
| Implementation risk | Already deployed and tested | Requires camera calibration, model conversion, topic plumbing |
| Failure modes | High-noise outdoor sun (long range); glass panels | Direct sun, lens flare, HDR scenes, unknown object classes |

## B.3 The Outdoor Morning Sun Scenario

A common question is whether camera-based detection helps in an open outdoor field, particularly in morning sun conditions. The answer is the opposite of the intuition: **morning sun makes camera-based detection worse, not better**.

### Effect on the lidar

- The Hesai lidar uses near-infrared laser pulses (905 nm). Direct sunlight raises the background IR noise floor, which can increase false-positive returns at *long range* (>10 m)
- At the short ranges used by the Collision Monitor (0.35–0.50 m), reflected laser pulses from solid obstacles are orders of magnitude stronger than solar noise — the obstacle is detected just as reliably as indoors

• The lidar is completely insensitive to sun direction

### Effect on a camera / YOLOv8

• **Sun facing the camera**: lens flare + full sensor overexposure — detector confidence collapses to near zero for the entire frame

• **Sun behind the camera**: obstacles are in deep shadow against a bright sky — low contrast, YOLO struggles with under-lit foreground objects

• **HDR scene** (bright sky + dark ground): sensor cannot expose correctly for both simultaneously; standard camera ISP clips one or the other

• YOLOv8n was trained on balanced-lighting datasets. Direct outdoor sun is one of its worst-case input distributions

WARNING: In an open field at low sun angle, a camera-based obstacle detector is least reliable precisely when you are most likely to need it. The lidar is unaffected by this condition.

## B.4 The Narrow Case Where YOLO Would Add Value

YOLOv8 is worth adding only if testing demonstrates that the current lidar-based system provably misses obstacles that appear in the competition. The specific cases where lidar fails:

• **Transparent obstacles** (glass panels, clear acrylic sheets): laser passes through, zero points returned, Collision Monitor is blind. Camera sees the object visually.

• **Very thin obstacles** (single thin pole, taut wire): may return fewer than max_points: 3 at certain angles, slipping past the threshold.

• **Above-lidar overhangs**: the height filter (0.05–1.5 m) means an obstacle above 1.5 m is ignored. A downward-facing camera could catch a low ceiling.

Unless the competition arena contains glass panels or sub-3-point thin obstacles confirmed through a test run, none of these cases justify the implementation cost.

## B.5 Decision Rule

Use the following rule when deciding whether to add YOLOv8:

• Run the robot through the actual competition course (or a representative mock-up)

• Review the rosbag: check if any obstacle was approached without the Collision Monitor triggering a stop or slowdown

• If yes — identify whether the missed obstacle is glass, a thin pole, or a class that YOLOv8 can detect reliably. If so, add the detector node.

• If no missed obstacles — the current stack is sufficient. Do not add YOLO.

NOTE: For the vast majority of competition run conditions — indoor arenas, outdoor courses with standard solid obstacles, any lighting — the lidar Collision Monitor is the correct and sufficient approach. The current software stack is competition-ready as-is. Invest available time in tuning nav2_params.yaml and running practice laps, not in adding camera infrastructure.